Design and Implementation of Distributed Applications
dida-tkvs

Gonçalo Rua, João Gouveia, Francisco Salgueiro
Instituto Superior Técnico
Av. Rovisco Pais 1, 1049-001 Lisboa, Portugal
{goncalonrua,joaotalonegouveia,francisco.salgueiro}@tecnico.ulisboa.pt

Abstract

dida-tkvs is a replicated key-value store composed of multiple servers, transactional clients, and a configuration console. High-performance consistency across replicas is achieved using the Multi-Paxos consensus algorithm.

1. Introduction

In this project, we implemented a transactional key-value store replicated using Paxos, to ensure that all servers maintain a consistent view of the data.

This paper highlights the key components of the system, focusing on transaction execution, the Multi-Paxos implementation, and how reconfiguration is supported.

2. Design

We developed all steps outlined in the guidelines. As such, our key-value store uses Multi-Paxos to decide the ID of the next transaction to be processed, while also correctly supporting reconfiguration.

2.1. Executing Transactions

Every server maintains a list of transactions that have not yet been executed locally and a priority queue of unprocessed request IDs, sorted by their timestamps. Additionally, each server keeps track of the next timestamp it expects to execute, which may be interpreted as the server's current position in history.

In order to execute a transaction, two conditions must be met:

1. The next timestamp the server expects to execute must match the timestamp of the ID at the top of the priority queue.
2. The transaction with that ID must be present in the server's list of transactions.

Whenever these two conditions hold true, the server executes the corresponding transaction, replies to the client with the result, removes the transaction from the list, and pops the ID from the queue.

The `MainLoop` class is where the aforementioned logic is handled. The `ServerState` class is where the list, priority queue, and the current position in history are stored.

2.2. Multi-Paxos

Consistency across replicas is ensured by using Multi-Paxos to decide the ID of the next transaction to be processed, effectively attributing a timestamp to each request.

Whenever the leader replica finishes using the previous batch to decide request IDs, it prepares a new batch of three Paxos instances. If, during this preparation phase, the leader encounters an instance in which some replicas have already voted for a value, it realizes that it is behind. Consequently, it immediately executes phase two of Paxos for that same instance in order to update itself.

Whenever a server receives a learn request, it compares the request's index to the next timestamp it expects to execute. If the request's index is less than the next expected timestamp, the server discards the learn request, as it refers to an ID that has already been processed locally. Otherwise, the server stores the ID in its priority queue. This approach may lead to storing duplicate request IDs. Duplicate ID removal is handled locally by each server.

Our implementation of Multi-Paxos differs from Lamport's description [1] only in how it handles the occurrence of gaps in the command sequence.

In our implementation, the leader keeps track of the number of the next Paxos instance it will use to propose an ID. If that Paxos instance has already been used to decide an ID, the leader skips ahead until it finds an instance number that has not yet been used.

Our strategy is obviously less efficient than Lamport's approach of using "no-op" commands to skip holes in the command sequence. Our naive approach is, however, much simpler to implement. This was the driving force behind our decision.

The mapping between what we just described and the corresponding code is analogous to how we handled executing transactions. The functionality described above is implemented the `MainLoop` class, while the auxiliary data is stored in the `ServerState` class.

2.3. Reconfiguration

To support reconfiguration, we implemented the strategy Lamport refers to as "Reconfiguration Made Simple" [2]. We chose this approach because, as the names suggest, it is easier to implement than its counterpart, "Reconfiguration Made Harder".

3. Conclusion

We implemented a robust and high-performance transactional key-value store, that uses Multi-Paxos to ensure consistency across replicas and supports reconfiguration.

The algorithms we used deviate very little from those described by Lamport in his original work.

In terms of code, the `MainLoop` class handles most of the logic regarding transaction execution and Multi-Paxos management. The `ServerState` class stores all the necessary data.

References

[1] L. Lamport. Paxos made simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)*, pages 51–58, Dec. 2001.

[2] L. Lamport, D. Malkhi, and L. Zhou. Reconfiguring a state machine. *ACM SIGACT News*, 41(1):63–73, Mar. 2010.